

单元测试一 (2025春)

题量: 12 满分: 100.0

 智能分析

94.0 分

考试时间: 2025-03-23 12:30 至 2025-03-23 13:30

一. 单选题 (共 6 题, 36.0 分)

1. (单选题, 6.0 分)

对于某循环双向链表, 现在需要删除curr指针所指的结点, 可以完成删除操作代码是 () 。

 AI讲解

A.

```
if(curr == NULL )
    return;
Link *tmp=curr;
curr->prev->next=curr->next;
curr->next->prev=curr->prev;
delete tmp;
```

B.

```
if(curr->next != NULL )
    return;
Link *tmp=curr;
curr->prev->next=curr->next;
curr->next->prev=curr;
delete tmp;
```

C.

```
if(curr == NULL )
    return;
Link *tmp=curr->next;
curr->prev->next=curr->next;
curr->next->prev=curr->prev;
delete tmp;
```

D.

```
if(curr->next != NULL )
    return;
Link *tmp=curr->next;
curr->next->prev=curr;
curr->next=curr->next->next;
```

2. (单选题, 6.0 分)

栈结构的应用不包括 ()。

AI 讲解

- A.
进制转换
- B.
括号匹配
- C.
缓冲区
- D.
递归

我的答案:C

正确答案:C



6.0 分

知识点:

3. (单选题, 6.0 分)

一个栈的输入序列为: 1,2,3,4,5,6, 若在进栈操作时允许出栈操作, 则栈的可能输出的序列是 ()。

AI 讲解

- A.
634215
- B.
346215
- C.
154362
- D.
432615

我的答案:C

正确答案:C



6.0 分

知识点:

4. (单选题, 6.0 分)

AI讲解

若用一个大小为6的数组来实现循环队列，且当rear和front的值分别为4，1。当从队列中删除两个元素，再加入三个元素后，rear和front的值分别为（ ）。

- A.
0和4
- B.
1和3
- C.
5和1
- D.
3和1

我的答案:B

正确答案:B



6.0 分

知识点:

5. (单选题, 6.0 分)

AI讲解

以下程序段的时间复杂度是多少？

```
for (int i = 1; i < n; i++) {  
    for (int j = 1; j < n; j *= 2) {  
        for (int k = 1; k < n; k++) {  
            std::cout << i + j + k << " ";  
        }  
    }  
}
```

- A.
 $O(n^2)$
- B.
 $O(n^2 \log n)$
- C.
 $O(n^3)$
- D.
 $O(n \log n)$

我的答案:B

正确答案:B



6.0 分

答案解析: 外层循环n次, 中层循环 $\log_2 n$ 次, 内层循环n次, 时间复杂度为 $O(n^2 \log n)$

知识点:

6. (单选题, 6.0 分)

关于字符串的操作, 以下哪个描述是不正确的?()

AI 讲解

A.
字符串的插入操作可以在任意位置插入一个字符或子串

B.
字符串的删除操作可以删除任意位置的字符或子串

C.
字符串的替换操作可以将指定子串替换为另一个子串

D.
字符串的拼接操作可以将字符串的长度减小

我的答案:D

正确答案:D



6.0 分

知识点:

二. 判断题 (共 4 题, 24.0 分)

7. (判断题, 6.0 分)

我们通常使用队列来处理函数或过程调用。

AI 讲解

A. 对

B. 错

我的答案:错

正确答案:错

6.0 分

知识点:

8. (判断题, 6.0 分)

因为链表为链式存储结构，而顺序表采用顺序存储结构，所以链表其添加和删除操作的效率比顺序表的效率要高。

AI 讲解

A. 对

B. 错

我的答案:错

正确答案:对

0.0 分

知识点:

9. (判断题, 6.0 分)

线性表的顺序存储结构在插入和删除操作时，时间复杂度总是 $O(1)$ 。

AI 讲解

A. 对

B. 错

我的答案:错

正确答案:错

6.0 分

答案解析: 顺序存储结构在插入和删除操作时，可能需要移动大量元素，时间复杂度为 $O(n)$ 。

知识点:

10. (判断题, 6.0 分)

在链式存储的线性表中，合并两个有序链表的时间复杂度是 $O(n)$ ，其中 n 是两个链表中更长的链表长度。

AI 讲解

A. 对

B. 错

我的答案:错

正确答案:错

6.0 分

答案解析: 需要看采用的哪种链式存储，如果需要合并两个有表头指针的双向循环链表，合并时需要时间复杂度为常数，即 $O(1)$ 。

三. 简答题 (共 1 题, 15.0 分)

11. (简答题, 15.0 分)

阅读下面程序段, 并回答问题:

- 1、该程序段的功能是什么?
- 2、分析该程序段的时间复杂度和空间复杂度。

```
void exam(int arr[], int n) {  
    for (int i = 0; i < n / 2; i++) {  
        swap(arr[i], arr[n - i - 1]);  
    }  
}
```

AI讲解

我的答案:

15 分

1. 功能是反转数组, 将原数组的前半部分与后半部分依次对调, 使整个数组元素顺序被逆置

11.

(1) 功能为反转数组, 将数组元素逆序输出.

(2) 时间复杂度为 $O(n)$.

循环执行 $\frac{n}{2}$ 次, 交换元素为常数时间 $O(1)$.

时间复杂度为 $O(\frac{n}{2}) = O(n)$

空间复杂度为 $O(1)$.

程序使用固定数量的临时变量, 空间复杂度为常数级.

2.时间复杂度为 $O(n)$:循环执行 $n/2$ 次, 每次交换元素为常数时间 $O(1)$,时间复杂度为 $O(n/2)=O(n)$
空间复杂度为 $O(1)$, 程序仅使用固定数量的临时变量, 空间复杂度为常数级。

正确答案:

功能: 将数组 arr 原地逆序。

时间复杂度: $O(n)$, 其中 n 是数组的长度。需要遍历数组的一半。

空间复杂度: $O(1)$, 仅使用了常数个额外变量。

答案解析: reverseArray

知识点:

四. 算法设计 (共 1 题, 25.0 分)

12. (算法设计, 25.0 分)

【算法设计题】

假设以1和0分别表示入栈和出栈操作。栈的初态和终态均为空, 入栈和出栈的操作序列可表示为仅由1和0组成的序列, 称可以操作的序列为合法序列, 否则称为非法序列。写出一个算法, 判定所给的操作序列是否合法。若合法, 返回1, 否则返回0。

例如: 11111, 为非法序列, 返回0;

11100, 为非法序列, 返回0;

10110100, 为合法序列, 返回1。

(1) 给出算法的基本设计思想。

(2) 根据设计思想, 采用C或C++语言描述算法, 关键之处给出注释。

(3) 分析算法的时间复杂度。

要求: 在纸上手写并签名, 拍照上交。

照片要清晰, 大小合适。

AI讲解

1) 算法基本设计思想

合法序列要满足以下条件

① 终态栈为空, 所以 0 和 1 的个数要相等

② 每次 0 操作前必须有 1 操作

③ 任何时刻 0 的数量小于等于 1 的数量

所以只有这三个条件同时满足才能判断序列合法。维护计数器 count 表示栈中元素数量

(2) 入栈 count 加一, 出栈 (遇到 0) count 减一; 若 count < 0, 则 0 的数量 > 1 的数量, 非法

(3) 若遍历结束 count == 0 则合法 否则非法

(2) 代码与注释

```
bool isValid (const string & sequence)
```

```
{  
    int count = 0; // 初始化计数器为 0
```

```
    for (char action: sequence) // 读入序列
```

```
        if (action == '1')
```

```
        {  
            count++; // 若读入 '1' 为入栈, 计数加 1
```

```
        }  
        else if (action == '0')
```

```
        {  
            count--; // 读入 '0' 为出栈, 计数减 1
```

```
            if (count < 0)
```

```
            {  
                return 0; // 若出栈次数大于入栈次数, 序列非法
```

```
            }
```

```
        }  
        else { return false; // 判断输入的是否为非法字符
```

```
        }
```

```
    }
```

```
    return count == 0; // 只有终态栈中元素为 0 才合法
```

(3) 时间复杂度: $O(n)$. n 为操作序列长度, 只要一次线性遍历,

每个字符处理时间为常数; 故时间复杂度为 $O(n)$